## Semana 3 · Temporizadores

*Unidad 1 · reloj, contador, ticks, prescaler y generación de señales*

### 1. Temporizador: concepto

Temporizador -> circuito/periférico que cuenta pulsos de reloj para medir un intervalo. En vez de calcular el tiempo a ojo, usa ticks.

↳ Yo lo veo como un contador que avanza usando pulsos de reloj. Cuando llega a cierta cantidad, ya pasó el intervalo configurado y se puede ejecutar una acción.

Reloj -> pulsos. Contador -> guarda cuántos pulsos han ocurrido.

Tick -> una unidad de avance del contador.

Alarma del timer -> cuando llega al valor configurado -> se genera un aviso/evento.

Timer -> repetitivo o de una sola vez, dependiendo de la configuración.

- Uso -> parpadeos periódicos, tareas cada cierto tiempo, mediciones, alarmas.

Nota al margen: En las hojas: reloj fijo + divisor (prescaler) -> resolución...

### 2. Frecuencia, periodo y resolución

Frecuencia (f) -> cantidad de pulsos por segundo, en Hz.

↳ Si aumenta la frecuencia hay más pulsos en un segundo y cada periodo es más corto. Por eso en los ejercicios paso de Hz a segundos antes de calcular ticks.

Periodo (T) -> cuánto dura un pulso, en segundos.

Fórmula -> $T = 1 / f$; también $f = 1 / T$.

1 MHz = 1 000 000 Hz. 1 ms = 0,001 s. 1 µs = 0,000001 s.

Resolución del temporizador -> duración de cada tick; determina el paso mínimo de tiempo que puede contar en esa configuración.

### Preescalador / prescaler

Preescalador -> divide la frecuencia de reloj para que el contador avance más despacio.

↳ El prescaler sirve para reducir la frecuencia con la que cuenta el timer. Así puedo medir intervalos más largos sin necesitar un contador tan grande.

$f_{tick} = f_{reloj} / divisor$.

$T_{tick} = 1 / f_{tick}$.

ticks necesarios = tiempo deseado / $T_{tick}$.

- Divisor mayor -> menos ticks por segundo -> cada tick dura más.

### 3. Cálculos parecidos a los del cuaderno

### Ejemplo 1: 1 MHz

↳ Para comprobar mi cálculo multiplico los ticks por el tiempo que dura uno. Si obtengo el intervalo esperado, la cuenta está consistente.

$f_{tick}$ = 1 000 000 Hz -> $T_{tick} = 1 / 1\,000\,000$ = 1 µs.

Quiero 0,5 s -> 0,5 / 0,000001 = 500 000 ticks.

Comprobación -> 500 000 × 1 µs = 0,5 s.

### Ejemplo 2: usar un divisor

Supongamos un reloj de 80 MHz y divisor 80:

80 000 000 / 80 = 1 000 000 Hz -> 1 tick cada 1 µs.

Para 10 ms -> 10 000 ticks.

**Ejemplo 3: 2 500 Hz**

T = 1 / 2500 = 0,0004 s = 0,4 ms.

Para 10 ms -> 10 / 0,4 = 25 ticks.

Nota: son ejercicios de cálculo. La frecuencia del timer real depende del reloj y la configuración utilizada en el ESP32.

**4. Delay y millis (no son iguales)**

delay(500) -> espera 500 ms; durante esa espera el flujo principal no sigue al siguiente paso.

millis() -> cuenta tiempo desde que inicia el programa; permite revisar si ya pasó un intervalo.

No bloqueante -> puedo revisar botones o hacer otras tareas mientras espero el tiempo.

```
unsigned long anterior = 0;
const unsigned long intervalo = 500;

void loop() {
  if (millis() - anterior >= intervalo) {
    anterior = millis();
    // aquí se cambia el estado del LED
  }
  // aquí se puede revisar otra entrada
}
```

- La resta entre tiempos evita depender de que millis() valga exactamente un número concreto.
- unsigned long -> tipo adecuado para guardar contadores de milisegundos en este ejemplo.

**5. Temporizador de hardware y señal PWM**

Timer de hardware -> periférico que sigue contando por separado de las instrucciones normales del programa. Puede generar un evento de interrupción.

Reloj -> prescaler -> contador -> alarma -> acción.

PWM -> señal digital que cambia rápido entre HIGH y LOW.

↳ PWM no baja necesariamente el voltaje de cada pulso; cambia la proporción de tiempo en alto y bajo. En un LED puede variar el brillo aparente.

Duty cycle -> porcentaje de tiempo encendido dentro de un periodo.

50 % -> mitad ON / mitad OFF. 25 % -> ON un cuarto del periodo.

Más duty -> mayor valor promedio de la señal en una carga adecuada; en LED suele verse más brillo.

PWM no produce por sí misma un voltaje analógico constante. Es una secuencia de pulsos.

## 6. Práctica / señal del temporizador (esquema)

Objetivo -> producir un cambio periódico en un GPIO y observar el periodo.

1. Elegir tiempo de encendido y apagado.
2. Calcular ticks con la frecuencia y prescaler.
3. Configurar la salida y temporizador.
4. Comparar el tiempo calculado con el comportamiento de la señal.

- Si el periodo aumenta -> frecuencia disminuye. Si el periodo baja -> frecuencia sube.
- No usar esperas largas dentro de una rutina de interrupción del timer.

Nota al margen: Si pide una señal de 1 Hz -> periodo 1 s. 50 % -> 0,5 s ON y 0,5 s OFF...

**Recordatorio**

Timer = cuenta. Tick = paso. Prescaler = divide. Alarma = avisa. ISR = atiende evento.